



Aletheia

Automated test reports for power transformers
CPRI Short Circuit Laboratory · Hackathon Track 3

In one line: Aletheia turns scattered lab paperwork into a checked, numbered, tamper-evident test report that the customer can verify, and it never claims a pass it can't back up.

The problem

When the CPRI Short Circuit Laboratory tests a power transformer, the results end up spread across about ten documents: the customer request, a work instruction, a proforma, and seven handwritten log sheets (losses, winding resistance, no-load current, routine tests, short circuit, temperature rise, oil-pressure test). Today an engineer copies those values into a report by hand, checks the arithmetic, compares results against the standards (IS 1180 and IS 2026) and sends a PDF. That is slow, mistakes are easy to make, and a customer can't check whether the PDF they received is genuine.

Aletheia does that whole process in one web app:

1 Request	2 Import	3 Validate	4 Generate	5 Approve	6 Verify
--------------	-------------	---------------	---------------	--------------	-------------

1. Getting data in, from whatever the lab already has

Real labs have scattered data, so the app accepts all the usual formats:

Source	What happens
CSV / TSV	Comma, semicolon or tab separated, any line endings (including files saved by Excel on Windows)
Excel (.xlsx)	One sheet for everything, or one sheet per document
Existing SQLite databases	Opened read-only. Any table in the section, field, value layout is picked up
JSON	The original format
Scans and photos (PDF, PNG, JPG)	Kept as evidence with a SHA-256 fingerprint

- **Exports load back in unchanged.** Any job can be downloaded as CSV, Excel, database or JSON, and that file imports again as it is. There are blank templates for each format too.
- **Nothing gets imported twice by accident.** The same content can't be added to one job twice. Several files can be dropped at once.
- **Imports can be undone.** Each import remembers which documents it brought in, so *Remove* takes exactly those back out.
- **A form page for every document,** for typing in or correcting values by hand.
- **New requests** can come from a form (which checks the ID formats), from a data file, or from a scan of the request form.

Optional AI scan reader

Click **Scan** on an attached sheet and a vision model reads it. You can use Qwen 2.5-VL running offline on the laptop through Ollama (so lab data never leaves the machine), any OpenAI-compatible hosted model (e.g. through OpenRouter), or Google Gemini.

- **Page clean-up.** Sideways pages are turned upright and converted to a high-contrast image.
- **Big sheets in parts.** Header fields first, then each table, so the form fills in as the model answers.
- **A fixed answer shape.** The model is held to a fixed JSON layout, which stopped it looping on large tables.
- **Uncertain fields are highlighted.** Nothing is saved until the engineer has reviewed it.
- **No copying from the answers.** The model is given an empty layout, never the sample's real values. An early version that did see real values just copied them back and looked perfect; that was caught and fixed.
- **Saved readings.** Every reading is stored by scan fingerprint, so the same sheet reads instantly and offline the second time.
- **A daily request cap** for hosted models keeps costs under control.

Measured accuracy: on a laptop CPU, the small 3B model gets about half the fields right and takes 20–45 seconds per short sheet, so every reading has to be checked by hand. The AI is an optional speed-up; the app works fully without it.

2. Validation: 30 engineering checks

These checks are the core of the app. They fall into five groups:

- **Arithmetic re-computation.** Logged averages and sums are recalculated from the raw readings.
- **Cross-document ID consistency.** The series and sample numbers must match on every sheet. The checker allows for handwriting confusions such as H/4, and flags 4/6.
- **Limits from the standards:** winding-resistance imbalance; no-load current at rated voltage and at 112.5% (depending on the rating); losses against the guaranteed values; impedance and voltage-ratio tolerances at each tap; short-circuit current, reactance change and thermal duration; top-oil and winding temperature rise; steady-state rate; the oil-pressure test.
- **Per-job limits.** Temperature-rise limits are read from that job's proforma rather than hardcoded, because they vary.
- **Thin margins.** A result only just inside its limit (for example a 0.3 K winding-rise margin) is flagged.

Each finding gets one of four results:

PASS	The check ran and the value is within its limit.
REVIEW	Doubtful but not a failure. The engineer marks it reviewed.
DATA ERROR	The data itself is wrong. This blocks the report until it is corrected.
REQUIREMENT NOT MET	A real test failure. The engineer confirms it, and the report states the sample does not comply , with a table of what failed.

Honest reporting: if a document is missing or a value was left empty, that test reads **NOT EVALUATED** or **NOT FULLY EVALUATED**, never PASS. The statement of conformity lists everything that was left out.

Every threshold has a source: each one lives in *rules.py* with the document it came from and a status. *docs/VALIDITY.md* lists what has and hasn't been confirmed against the standards.

3. Report generation

- **A numbered, unchangeable report.** The PDF is filled from the database and stored with a version number and SHA-256 fingerprint. Once stored, a version is never rebuilt.
- **Wording can be changed without a restart.** The organisation name, title, headings, signature labels and footer are all in *report_template.json*, which is read each time a report is built.
- **The report always matches the data.** Changing any test data sends the job back to "Data imported", so a stale report can't be released.

4. Review, approval and release controls

- Preview, print or download the report.
- **Approval needs a name and an employee ID**, and both are printed on the report.
- **The test engineer named on the report cannot approve it**, so someone else always has to sign off.
- **An optional approver list** (ALETHEIA_APPROVERS) limits approval to specific employee IDs.
- **A released report can't be deleted**, so the customer's QR code keeps working. It can be withdrawn instead, and it then shows as withdrawn when checked.
- Editing an approved record sends it back for re-approval.
- **Every step is recorded in a time-stamped history.**

5. Customer verification

Every PDF carries a **QR code** that opens a verification page. That page recomputes the fingerprint and tells the customer whether this exact PDF is **intact**, whether it is the **current version**, and whether it is **approved**. The customer can download the approved PDF from the same page. This makes forged or altered reports detectable.

6. Day-to-day use

- **Dashboard:** how many jobs are at each stage; average turnaround from request to release (measured from the history); the oldest open job; the quality gate; charts of results against their limits; recent reports with their results; AI usage.
- **Report Workflow:** a live pipeline of jobs waiting, oldest first, with how long each has waited.
- **Records & Search:** full-text search over series, sample, customer, rating, tests, standard, engineer, approver and dates; filters for status, result (complies / does not comply / not yet checked), date range, and historical records.
- **Importing old registers:** the lab's existing record of past tests (CSV, Excel or SQLite) becomes searchable historical records. Common header variants are recognised ("Test Series No", "Date of Test"); results such as "Passed" or "Failed – ..." are read as complies / does not comply; dates are normalised; invalid or duplicate rows are listed with the reason they were skipped.
- **Help assistant:** a built-in chat assistant that starts a new request, searches records, gives a job's status and answers how-to questions. It works by keyword matching, with no AI, so it always works offline.
- **Architecture page:** a diagram of the system inside the app itself.

7. Engineering quality

- **60 automated tests** check that exported data loads back in exactly; CSV, Excel and database imports give the same result as JSON; a full run from CSV to an approved report works; failing samples get "does not comply" reports; data errors block the report; the release rules hold; tampered reports are detected.
- **One "mutation test" per engineering check:** a single fault is injected and the test confirms that exactly the right finding changes. This proves each check catches what it is meant to catch.
- **Small and simple to run:** Python with Flask, SQLite, ReportLab and openpyxl. *pip install* and *python app.py*, and it runs offline on one laptop.

8. Limits, stated openly

- **The thresholds still need a lab engineer.** None has been confirmed by a lab engineer yet; each is marked *secondary* (matches a published summary) or *unconfirmed*.
- **No user accounts.** The approver is a typed name and employee ID, so the app should run on a trusted network.
- **Handwriting is still the slowest step.** The AI reader helps but needs human checking.
- **This is a hackathon prototype, not an official CPRI system.**

Live demo (about 2 minutes)

Step	What to show
1	Try the demo data → Run checks → mark each flagged item as reviewed → Generate report
2	Approve with a name other than the test engineer (P. Naveenkumar). The app refuses the engineer's own name, which is worth showing.
3	Copy the customer link → open the verification page and download the approved PDF.
4	Failing case: drop <i>test-files/3 - failing job (top-oil rise over limit).csv</i> . The report says DOES NOT COMPLY .
5	Missing data: drop <i>test-files/2 - partial job (8 documents, 3 values empty).csv</i> . The affected tests read NOT EVALUATED , never PASS.